

Junior Santiago Daza Rodriguez

En la clase `Datos.Java`: // Viola el patrón 6, se usó **public static** en toda la clase, lo cual **viola a las estructuras de proyectos limpias** porque no valora el encapsulamiento, **rompiendo el principio de ocultación de información** y entonces se comporta como una bolsa de variables globales que pueden causar bugs difíciles de rastrear, además el nombre de **Datos** es muy genérico, se podría cambiar **a un modelo de objetos real, por ejemplo una Clase Productos con atributos nombre y precio y usar una lista, además los atributos deberían ser privados y accederse mediante métodos controlados.**

```
Line : public static String[] nom={"Bandeja Paisa","Sancocho de Gallina","Arepa con Huevo","Jugo Natural","Gaseosa","Cerveza Poker","Agua Panela","Arroz con Pollo"};
```

//Viola el patrón 1, se usó un **nombre extremadamente corto** que no describe su propósito, cambiar a **nombreProductos**.

// Viola el patrón 4, **el código está amontonado, lo vuelve difícil de leer, separarlo por bloques e identarlo.**

// Viola el patrón 8, **Harcodea valores**, los nombres de los platos están directamente en el código, es decir, si el restaurante quiere cambiar o agregar un plato hay que modificar el código fuente y recompilar, cambiar a **uso de archivos o definirse como mínimo, como constantes si serán fijos.**

```
Line : public static double[]  
p={32000,28000,8000,7000,4500,6000,3500,25000};
```

// Viola el patrón 1, se usó un **nombre extremadamente corto** que no describe el propósito, cambiar a **precios**.

// Viola el patrón 8, **Harcodea valores**, los valores de los platos están directamente en el código, es decir, si el restaurante quiere cambiar o agregar un valor a un plato hay que modificar el código fuente y recompilar, cambiar a **uso de archivos o definirse como mínimo, como constantes si serán fijos**.

Line : public static int[] **cant**={0,0,0,0,0,0,0,0};

//Viola el patrón 1, se uso un **nombre extremadamente corto** que no describe el propósito, cambiar a **cantidades**.

// Viola el patrón 4, **el código está amontonado**, lo vuelve difícil de leer, separarlo por espacios.

Line 15: public static int **ms**=0;public static int **est**=0;

//Viola el patrón 1, se usaron nombres extremadamente cortos que no describen su propósito, cambiar a **numeroMesa y estadoMesa**.

//Viola el patrón 2, **tiene declarada múltiples variables en una sola línea** y tiene faltas de espacios.

// Viola el patrón 4, **el código está amontonado**, lo vuelve difícil de leer, separarlo por bloques e indentarlo.

public static int **ms**=0;public static int **est**=0;

La práctica de Código limpio indica que **debe tener cada una su propia línea de código** y que exista **espacios alrededor de los operadores** para reducir el esfuerzo de lectura

Line : public static double **tot**=0;public static int **nf**=1;

//Viola el patrón 1, se usó un **nombre extremadamente corto** que no describe su propósito, cambiar a **totalPedido y numeroFactura**.

// Viola el patrón 4, **el código está amontonado**, lo vuelve difícil de leer, separarlo por bloques e indentarlo.

```
Line : public static String x="El Buen Sabor";
```

//Viola el patrón 1, se usaron **nombre extremadamente corto**
que no describen su propósito, cambiar a **texto**

```
Line : public static String tmp="";
```

//Viola el patrón 1, se usaron **nombre extremadamente corto**
que no describen su propósito, cambiar a **cadenaTemporal**.

En la clase `Imprimir.Java`:

```
Line : public static void mostrarPedido(){
double sub=0;int i=0;
System.out.println("--- PEDIDO ACTUAL ---");
while(i<Datos.nom.length){
if(Datos.cant[i]>0){
// imprime producto con cantidad y subtotal parcial
System.out.printf("%-20s x%-6d
%,.0f%n",Datos.nom[i],Datos.cant[i],(Datos.p[i]*Datos.cant[i]));
// suma al subtotal
Line 30: sub=sub+Datos.p[i]*Datos.cant[i];}
i++;} // fin while
System.out.println("-----");
System.out.printf("%-27s $%,.0f%n","Subtotal:",sub);}
```

//Viola el patrón 2, **no mantiene las funciones con enfoque de precisión laser**, en vez de limitarse a imprimir hace cálculos complejos, el patrón SRP dicta que una función debe hacer una sola cosa y hacerla bien. Cambiar la función `mostrarPedido()` para que cumpla una única función con valores ya calculados como parámetros para mostrarlos.

```
Line : public static void imprimirFacturaCompleta(){
double sub=0;double iva=0;double tot=0;double prop=0;int
cont=0;double aux=0;
// calcula subtotal otra vez
int i=0;while(i<Datos.nom.length){
```

```

if(Datos.cant[i]>0){
    sub=sub+Datos.p[i]*Datos.cant[i];cont=cont+1;}
i++;} // fin while
if(cont>3){aux=sub-(sub*0.05);}else{aux=sub;}
if(aux>50000){
    iva=aux*0.19;
    tot=aux+iva;
    prop=tot*0.10;
    tot=tot+prop;}
else{
    iva=aux*0.19;
    tot=aux+iva;
    prop=0;} // fin if-else
String sep="===== ";
System.out.println(sep);
System.out.println("    RESTAURANTE EL BUEN SABOR");
System.out.println("    Calle 15 #8-32, Valledupar");
System.out.println("    NIT: 900.123.456-7");
System.out.println(sep);
System.out.printf("FACTURA No. %03d%n",Datos.nf);
System.out.println("-----
");
// imprime cada item del pedido
int j=0;while(j<Datos.nom.length){
    if(Datos.cant[j]>0){
        System.out.printf("%-20s x%-6d
        $%,.0f%n",Datos.nom[j],Datos.cant[j],(Datos.p[j]*Datos.cant[j]
        ));}

```

```

j++;} // fin while

System.out.println("-----
");

System.out.printf("%-27s $%,.0f%n","Subtotal:",aux);

System.out.printf("%-27s $%,.0f%n","IVA (19%):",iva);

if(prop>0){

System.out.printf("%-27s $%,.0f%n","Propina (10%):",prop);} //
fin if prop

System.out.println("-----
");

System.out.printf("%-27s $%,.0f%n","TOTAL:",tot);

System.out.println(sep);

System.out.println("Gracias por su visita!");

System.out.println("El Buen Sabor - Valledupar");

System.out.println(sep);

// actualiza estado e incrementa factura - tres
responsabilidades en un metodo

Datos.nf=Datos.nf+1;

Datos.est=0;

Datos.tot=tot;}

// Viola el patrón 1, hay variables como, i,j,sub, cont, aux,
sep que no explican su función en el código, se pueden
renombrar a IndincePlato, CantidadPlatosDiferentes,
separaradorEstetico, subTotalBruto;

//Viola el patrón 2, no mantiene las funciones con enfoque de
precisión laser, en vez de limitarse a imprimir hace 3 cosas
en un solo metodo, el patrón SRP dicta que una función debe
hacer una sola cosa y hacerla bien. Cambiar la función

```

`imprimirFacturaCompleta()` para que cumpla una única función con valores ya calculados como parámetros para mostrarlos.

// Viola el patrón 4, **el código está amontonado**, lo vuelve difícil de leer, separarlo por bloques e indentarlo.

// Viola el patrón 7, **el código no mantiene la estructura de indentación**, lo vuelve difícil de leer, hacer valer el patrón 4.

// Viola el patrón 8, **Harcodea valores**, los valores de iva, propina, descuento y el umbral de 5000 están directamente en el código, **definir estos datos como constantes**, como `Porcentaje_IVA = 0.19`.

// Viola el patrón 9, **el código tiene funciones de todo menos breves**, lo vuelve difícil de leer, entender, reparar.

En: `public static void imprimirFacturaResumen(){`

`double sub=0;double iva=0;double tot=0;double prop=0;int cont=0;double aux=0;`

// Viola el patrón 1, **los nombres aux, cont, sub, iva, prop son abreviaturas sin significado claro**. Renombrar a: `subtotalBruto, cantidadProductosDiferentes, subtotalConDescuento, montoIva, montoPropina`.

// Viola el patrón 2 y 9, **imprimirFacturaResumen() duplica exactamente el mismo bloque de cálculo de imprimirFacturaCompleta()**. Extraer el cálculo a un método privado `calcularTotalesFactura()` y llamarlo desde ambas funciones.

// Viola el patrón 3, **el comentario "// calcula subtotal otra vez igual que en imprimirFacturaCompleta"** admite que es

código duplicado y aun así se deja así. Si detectas duplicación, refactorízala; un comentario que admite código repetido es una señal de alarma.

```
// Viola el patrón 8, los valores 0.19, 0.10, 0.05 y 50000
están hardcoded de nuevo en esta función. Definir
constantes como: PORCENTAJE_IVA = 0.19, PORCENTAJE_PROPINA =
0.10, DESCUENTO_ESPECIAL = 0.05, UMBRAL_PROPINA = 50000.

// Viola el patrón 9, la función tiene más de 30 líneas
haciendo cálculos, formato de cabecera, totales e impresión
de renglones. Dividir en métodos pequeños: calcularTotales(),
imprimirEncabezado(), imprimirRenglonesPedido(),
imprimirTotales().

// Viola el patrón 4 y 7, todo el código está comprimido sin
sangría ni espacios, mezclando declaraciones y lógica en una
línea. Aplicar indentación consistente de 4 espacios y
separar cada sentencia en su propia línea.
```

En la clase Proceso.Java:

```
Line : public static double hacerTodo(){
```

```
double sub=0;double iva=0;double tot=0;double aux=0;int
cont=0;int i=0;
```

```
// Viola el patrón 1, las variables iva, tot, aux, cont, i
son nombres de una sola letra o abreviaturas sin contexto.
Renombrar a: montoIva, totalFinal, subtotalConDescuento,
cantidadPlatosDiferentes, indicePlato.
```

```
// Viola el patrón 1, el nombre "hacerTodo" es el peor nombre
posible para una función: no describe QUÉ hace ni PARA QUÉ
sirve. Renombrar a calcularTotalPedidoConImpuestos() o
calcularMontoFinalFactura() para que el nombre revele la
intención.
```


// Viola el patrón 2 (SRP), `hacerTodo()` calcula subtotal, aplica descuento por cantidad, calcula IVA, calcula propina y actualiza estado global en Datos. Son 5 responsabilidades distintas en un único método. Separar en: `calcularSubtotal()`, `aplicarDescuento(sub, cant)`, `calcularIva(base)`, `calcularPropina(base)`, `actualizarEstado(total)`.

// Viola el patrón 3, el comentario "// version anterior - no borrar" indica código muerto comentado que nunca fue eliminado. Eliminar todo código comentado que no se usa; para eso existe el control de versiones (Git).

// Viola el patrón 4 y 7, sentencias como `'Datos.est=1;Datos.tot=tot;'` en una sola línea dificultan la lectura y el debugging. Cada asignación en su propia línea con indentación correcta.

// Viola el patrón 8, 0.05 (descuento), 0.19 (IVA), 0.10 (propina) y 50000 (umbral) son magic numbers directamente en el código. Declarar como constantes en una clase de configuración: `DESCUENTO_MULTI_PLATO = 0.05`, `IVA = 0.19`, `PROPINA = 0.10`, `UMBRAL_PROPINA = 50000.0`.

// Viola el patrón 9, `hacerTodo()` tiene más de 25 líneas con lógica condicional anidada a 3 niveles de profundidad, imposible de probar por separado. Cada cálculo debe ser una función independiente de 3-5 líneas que pueda probarse con JUnit de forma aislada.

// Viola el patrón 6, `hacerTodo()` modifica directamente `Datos.est` y `Datos.tot` (estado global), creando acoplamiento fuerte entre la lógica de negocio y los datos. La función debería retornar el total calculado y el llamador actualizaría el estado; así se separan responsabilidades.

```
Line : public static double procesar(double a, double b,  
double c, double d, double e, int f, boolean g){
```

// Viola el patrón 1 - CRÍTICO, todos los parámetros son letras sueltas (a, b, c, d, e, f, g) sin ningún significado. Es imposible saber qué representa cada argumento al llamar

este método. Renombrar: precioUnitario, cantidad, porcentajeDescuento, porcentajeIva, porcentajePropina, cantidadItems, aplicarPropina.

// Viola el patrón 2 (SRP), procesar() recibe 7 parámetros y realiza: multiplicación precio-cantidad, descuento, IVA, propina y bonificación por cantidad de ítems, todo mezclado. Una función con 7 parámetros es una señal clara de que hace demasiado; dividir en funciones especializadas con máximo 3 parámetros cada una.

// Viola el patrón 3, el comentario "// aplica descuento" describe el QUÉ, no el POR QUÉ. Si el código es claro, el comentario sobra. Usar nombres tan descriptivos que el comentario sea innecesario; reservar comentarios para explicar decisiones de negocio no obvias.

// Viola el patrón 4, la línea "iva=res*d;tmp=iva;" mezcla dos operaciones distintas sin espacio ni separación visual alguna. Escribir cada operación en su propia línea: montoIva = subtotal * porcentajeIva; aplicarIva al subtotal.

// Viola el patrón 8, el valor 0.01 en "res=res-(res*0.01)" es un magic number sin nombre ni explicación. ¿Qué regla de negocio representa?. Definir DESCUENTO_ADICIONAL_POR_ITEMS = 0.01 o mejor aún documentar la regla de negocio que lo justifica.

// Viola el patrón 5, un método con 7 parámetros y lógica condicional anidada es prácticamente imposible de cubrir con pruebas unitarias de forma exhaustiva. Al dividir en funciones pequeñas y bien nombradas, cada una puede tener su propio test en JUnit con casos: descuento con 0%, con 5%, IVA base, etc..

// Viola el patrón 9, la función tiene 7 parámetros y más de 15 líneas de lógica condicional. Una función larga con muchos parámetros es señal de que hace demasiadas cosas. Aplicar el principio: si necesitas más de 3 parámetros, probablemente debes crear un objeto que los encapsule (ej: ObjetoFactura).

En la clase RestauranteElBuenSabor.Java (método main):

Line : `Scanner sc=new Scanner(System.in);`

`int op=0;boolean flag=true;int x=0;String aux="";int
tmp=0;double m=0;boolean continuar=true;`

`// Viola el patrón 1, las variables flag, x, aux, tmp, m
carecen de nombres que expresen su propósito. ¿Qué es 'm'?
¿Qué bandera es 'flag'? Renombrar: menuActivo (boolean),
intentosInvalidos (int), mensajeTemporal (String),
totalCalculado (double). Eliminar variables que nunca se usan
como 'continuar' y 'x' que se reinician sin efecto real.`

`// Viola el patrón 2 (SRP), el método main() tiene más de 100
líneas manejando: presentación, menú, validación de entrada,
flujo de opciones, reinicio de estado, manejo de errores. Es
el anti-patrón de 'God Method'. Extraer métodos:
mostrarEncabezado(), mostrarMenu(), procesarOpcion(int op),
manejarAgregarProducto(), manejarGenerarFactura(),
manejarNuevaMesa().`

`// Viola el patrón 4 y 7, la indentación es inconsistente a
lo largo del método: algunas líneas tienen 4 espacios, otras
6, otras ninguno, lo que dificulta identificar los bloques
lógicos. Aplicar el formatter de IntelliJ o configurar Google
Java Format en el proyecto para que normalice automáticamente
todo el código.`

Line: `if(op==2){ // sección agregar producto`

`// Viola el patrón 4, la cadena if-else if-else de 80+ líneas
dentro del while principal crea una pirámide de código
anidado de 5 niveles de profundidad. Reemplazar con un
switch-case o extraer cada rama a un método independiente; la
regla es: si necesitas scrollar para ver el cierre de un if,
el bloque es demasiado largo.`

```
Line: if(Datos.ms>0){ Datos.est=1;
aux=String.valueOf(Datos.ms); tmp=Datos.ms; x=tmp+1; }
```

// Viola el patrón 1, la variable "x" se calcula como "tmp+1" pero su valor nunca se usa en ninguna lógica posterior; es una variable zombie. Eliminar variables que no contribuyen a ninguna lógica real; código muerto confunde a quien lee y hace pensar que tiene un propósito oculto.

// Viola el patrón 8, el valor "1" en "Datos.ms=1" como fallback cuando la mesa es inválida es un valor mágico sin explicación; tampoco se notifica al usuario que se usó mesa 1 por defecto. Definir MESA_POR_DEFECTO = 1 como constante y notificar al usuario explícitamente: 'Mesa inválida, se asignó la mesa 1 por defecto'.

```
Line: double r=0; r=Proceso.hacerTodo(); tmp=(int)r;
aux="Total calculado: $" +tmp; m=r;
```

// Viola el patrón 1, la variable "r" para almacenar el resultado del cálculo es un nombre de una letra sin contexto. ¿"r" de resultado, de retorno, de restaurant?. Renombrar a totalFactura o montoFinalPedido; el nombre debe eliminar toda ambigüedad.

// Viola el patrón 9, se calculan tmp=(int)r y aux="Total calculado: \$" +tmp pero estas variables nunca se imprimen ni usan en ningún lugar; son computaciones inútiles. Eliminar código que no contribuye al resultado final; cada línea de código tiene un costo de mantenimiento y debe justificar su existencia.

```
Line: Scanner sc2=new Scanner(System.in); ... String
s2=sc2.hasNextLine()?sc2.nextLine():"";
```

// Viola el patrón 2 (SRP) y patrón 9, se crea un segundo Scanner sc2 dentro del bloque de opción inválida. Instanciar

un Scanner dentro de un caso de error específico es innecesario, confuso y puede causar problemas de recursos no cerrados. Usar el mismo Scanner sc que ya existe desde el inicio del método. Nunca crear recursos (Scanner, conexiones) dentro de ramas condicionales de manejo de error.

// Viola el patrón 3, el comentario "// limpiar buffer con segundo scanner - innecesario" es el mismo código admitiendo que la línea es innecesaria y aun así permanece. Regla de oro: si escribes un comentario que dice que algo es innecesario, elimina ese algo inmediatamente.

// Viola el patrón 10 (Boy Scout Rule), el contador 'x' para los intentos inválidos se reinicia a 0 tras 3 intentos pero no tiene ningún efecto real sobre el programa (no cierra el menú ni bloquea el sistema). Si una funcionalidad no está completa, implementarla bien o eliminarla. El código a medias es más peligroso que no tenerlo porque genera falsas expectativas.

Resumen General de Violaciones por Patrón:

Patrón 1 - Nombres significativos: Prácticamente todas las clases tienen variables de 1-3 letras (x, m, r, i, j, ms, est, tot, nf, aux, tmp, sub, iva, cont). Ningún desarrollador nuevo puede entender el código sin ejecutarlo.

Patrón 2 - Responsabilidad única (SRP): hacerTodo(), imprimirFacturaCompleta(), procesar() y main() mezclan cálculos, presentación, persistencia de estado y lógica de negocio. Cada método hace entre 3 y 6 cosas distintas.

Patrón 3 - Comentarios con criterio: Hay código muerto comentado ('version anterior - no borrar'), comentarios que describen lo obvio ('// multiplica precio por cantidad') y comentarios que admiten problemas sin resolverlos.

Patrón 4 - Código legible: Múltiples declaraciones en una línea, ausencia de espacios alrededor de operadores, bloques if-else sin indentación consistente en todo el proyecto.

Patrón 6 - Estructura de proyecto: La clase Datos es una bolsa de variables estáticas globales sin encapsulamiento. No hay separación real entre modelo de datos, lógica de negocio y presentación.

Patrón 7 - Formato consistente: El estilo de indentación cambia entre clases; algunas usan 4 espacios, otras no tienen. Los operadores a veces tienen espacio y a veces no (x=0 vs x = 0).

Patrón 8 - Sin hardcoding: IVA (0.19), propina (0.10), descuento (0.05), umbral (50000), menú fijo con 8 platos y precios todos hardcodeados directamente en el código. Cualquier cambio requiere recompilar.

Patrón 9 - Funciones breves: hacerTodo() tiene 25+ líneas, imprimirFacturaCompleta() tiene 40+ líneas, main() tiene 100+ líneas. Ninguna función debería superar las 20 líneas.

Aplicar estos cambios convierte el sistema de restaurante en código mantenible, escalable y profesional, listo para ser extendido con nuevas funcionalidades sin romper lo que ya funciona.

